

torch.linalg.solve_triangular#

https://pytorch.org/docs/stable/generated/torch.linalg.solve_triangular.html

Description:

Computes the solution of a triangular system of linear equations with a unique solution. Letting $K \in \mathbb{R} \text{ or } \mathbb{C}$, this function computes the solution $X \in K^{n \times k}$ of the linear system associated to the triangular matrix $A \in K^{n \times n}$ without zeros on the diagonal (that is, it is invertible) and the rectangular matrix $B \in K^{n \times k}$, which is defined as The argument `upper` signals whether A is upper or lower triangular. If `upper=False`, this function returns the matrix $X \in K^{n \times k}$ that solves the system If `upper=True` (resp. `False`) just the upper (resp. lower) triangular half of A will be accessed. The elements below the main diagonal will be considered to be zero and will not be accessed.

Function Signature

```
torch.linalg.solve_triangular(A, B, *, upper, left=True,
unitriangular=False, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`upper` (bool) – whether `A` is an upper or lower triangular matrix. `left` (bool, optional) – whether to solve the system $AX=B$ or $XA=B$. Default: `True`. `unitriangular` (bool, optional) – if `True`, the diagonal elements of `A` are assumed to be all equal to 1. Default: `False`. `out` (Tensor, optional) – output tensor. `B` may be passed as `out` and the result is computed in-place on `B`. Ignored if `None`. Default: `None`.

Code Examples

```
>>> A = torch.randn(3, 3).triu_()
>>> B = torch.randn(3, 4)
>>> X = torch.linalg.solve_triangular(A, B, upper=True)
>>> torch.allclose(A @ X, B)
True

>>> A = torch.randn(2, 3, 3).tril_()
>>> B = torch.randn(2, 3, 4)
>>> X = torch.linalg.solve_triangular(A, B, upper=False)
>>> torch.allclose(A @ X, B)
True

>>> A = torch.randn(2, 4, 4).tril_()
>>> B = torch.randn(2, 3, 4)
>>> X = torch.linalg.solve_triangular(A, B, upper=False,
left=False)
>>> torch.allclose(X @ A, B)
True
```

Notes & Warnings

NOTE See also `torch.linalg.solve()` computes the solution of a general square system of linear equations with a unique solution.

Detailed Documentation

Documentation

Computes the solution of a triangular system of linear equations with a unique solution.

Letting $K \in \{\mathbb{R}, \mathbb{C}\}$, this function computes the solution $X \in K^{n \times k}$ of the linear system associated to the triangular matrix $A \in K^{n \times n}$ without zeros on the diagonal (that is, it is invertible) and the rectangular matrix $B \in K^{n \times k}$, which is defined as

The argument `upper` signals whether A is upper or lower triangular.

If `left = False`, this function returns the matrix $X \in K^{n \times k}$ that solves the system

If `upper = True` (resp. `False`) just the upper (resp. lower) triangular half of A will be accessed. The elements below the main diagonal will be considered to be zero and will not be accessed.

If `unitriangular = True`, the diagonal of A is assumed to be ones and will not be accessed.

The result may contain NaNs if the diagonal of A contains zeros or elements that are very close to zero and `unitriangular = False` (default) or if the input matrix has very small eigenvalues.

Supports inputs of `float`, `double`, `cfloat` and `cdouble` dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

`torch.linalg.solve()` computes the solution of a general square system of linear equations with a unique solution.

A (Tensor) – tensor of shape $(*, n, n)$ (or $(*, k, k)$ if `left= False`) where $*$ is zero or more batch dimensions.

B (Tensor) – right-hand side tensor of shape $(*, n, k)$.

upper (bool) – whether A is an upper or lower triangular matrix.

left (bool, optional) – whether to solve the system $AX=BAX=BAX=B$ or $XA=BXA = BXA=B$. Default: True.

unitriangular (bool, optional) – if True, the diagonal elements of A are assumed to be all equal to 1. Default: False.

out (Tensor, optional) – output tensor. B may be passed as out and the result is computed in-place on B. Ignored if None. Default: None.